

Dokumentácia k projektu: Klasifikácia obrázkov MNIST pomocou neurónovej siete

Bc. Hana Magyarová

1.12.2024

Contents

1	Úvod	2
2	Predbežné spracovanie údajov	2
3	Architektúra modelu	3
4	Dopredné šírenie a stratová funkcia	4
5	Spätné šírenie	5
6	Trénovanie modelu	5
6.1	Nastavenie parametrov	5
6.2	Priebeh tréovania	6
7	Vyhodnotenie	7
7.1	Otestovanie na vlastnom obrázku	7
8	Diskusia a zlepšenia	9
9	Záver	10

1 Úvod

Pracovali sme s datasetom fashion MNIST, obsahujúci 70 000 čiernobielych obrázkov s rozlíšením 28x28 pixelov. Dataset už bol prerozdelený na 10 000 testovacích dát a 60 000 trénovacích dát. Dataset celkovo obsahuje 10 kategórií oblečenia:

- 0 - t-shirt/top - tričko s krátkym rukávom
- 1 - trousers - nohavice
- 2 - pullover - tričko s dlhým rukávom
- 3 - dress - šaty
- 4 - coat - kabát
- 5 - sandal - sandále
- 6 - shirt - tričko
- 7 - sneaker - tenisky
- 8 - bag - taška
- 9 - ankle boot - členková topánka

Cieľom projektu bude navrhnuť a implementovať neurónovú sieť bez použitia frameworkov hlbokého učenia (pytorch, tensorflow...). Hlavnou podmienkou splnenia zadania, je dosiahnuť úspešnosť aspoň 50%.

2 Predbežné spracovanie údajov

Na odpoveď stačia 1-2 vety. Popíšte proces načítania a prípravy dát na trénovanie a vyhodnotenie modelu.

Keďže sme celý kód písali v jupyter notebook na portáli Kaggle, použili sme priamo dataset odtiaľ. Konkrétne sme pracovali s vyznačenými .csv súbormi, a následne sme vytvorili vlastný dataset mathai-tricko, odkiaľ sme načítali vlastný obrázok.

Pre spracovanie údajov na trénovanie sme ich najprv načítali do numpy poľa, a zároveň sme normalizovali ich hodnoty (0-1) predelením číslom 255. Transponovaním (riadky sme zmenili na stĺpce) sme zabezpečili, že každý obrázok (784 pixelov) bude reprezentovaný ako vektor stĺpcov vhodný ako vstup do modelu.

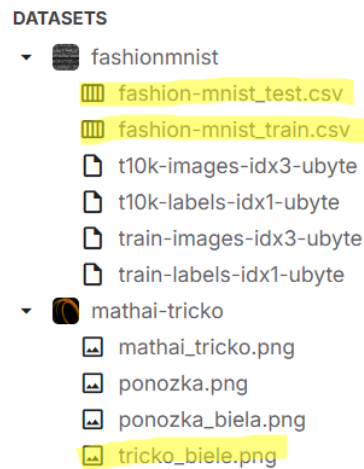


Figure 1: Použité datasety

3 Architektúra modelu

Pri spustení kódu je možné predefinovať si architektúru siete - teda počet skrytých vrstiev a počty neurónov. My sme vyhodnotili 3 skryté vrstvy s počtami neurónov 128, 64, 32 ako najefektívnejšie riešenie.

```

1 class NeuralNetwork:
2     def __init__(self, layers):
3         # Inicializacia vah (he inicializacia) a biasov
4
5     def forward(self, X):
6         # Dopredne sirenje - od vstupnej vrstvy po vystup
7         # Vstupna vrstva-> Skryta vrstva -> Vystupna vrstva
8
9     def compute_loss(self, y_pred, y_true):
10        # Pocitanie cross-entropy loss
11
12    def backward(self, X, y, learning_rate):
13        # Pocitanie gradientov pre spatne sirenje
14        # Aktualizacia vah a biasov
15
16    def train(self, X_train, y_train, learning_rate, patience
17              , min_delta)
18        # v trenovani vyuzivame early stopping -> koncime,
19        # ked sa po predefinovanom pocte epoch nezlepsi
20        # loss
21        # priebezne vypisujeme kontrolne vypisy kazdych 100
22        # epoch
23        # priebezne ratame aj train accuracy

```

```

21     def evaluate(self, X_test, y_test):
22         # Pocitanie accuracy modelu
23
24     # Inicializaia modelu s rozmermi pre vstupnu, skrytu a
        vystupnu vrstvu
25     if __name__ == "__main__":
26     # definicia vrstiev: input layer, 3 hidden layers, output
        layer
27     layers = [784, 128, 64, 32, 10]
28     model = NeuralNetwork(layers)
29     model.train(X_train, y_train, learning_rate=0.01, patience
        =20, min_delta=0.001)

```

Ďalšie pomocné funkcie ktoré sme použili:

```

1  def load_data(train_csv, test_csv):
2      # nactanie a predspracovanie dat
3
4  def relu_derivative(z):
5      # vrati deriváciu relu
6
7  def softmax(z):
8      # implementacia softmax funkcie pre vystupnu vrstvu
9
10 def relu(z):
11     # implementacia relu aktivacnej funkcie

```

4 Dopredné šírenie a stratová funkcia

V implementácii dopredného šírenia, vstupy (obrázky) postupne prechádzajú cez skryté vrstvy, kde sa počítajú hodnoty Z ako lineárna kombinácia váh, predchádzajúcich aktivácií a biasov ($Z = W * A + b$). Tieto hodnoty sa transformujú pomocou aktivačnej funkcie ReLU. Pre výstupnú vrstvu sa použije softmax funkcia, ktorá normalizuje hodnoty na pravdepodobnosti pre jednotlivé triedy; pričom ak sčítame všetky hodnoty, mali by sme dostať 1. Stratu počítame pomocou cross-entropy, ktorá porovnáva predikované pravdepodobnosti s reálnymi labels (one-hot enkódované), pričom sa počíta logaritmus pravdepodobností a tieto hodnoty sa následne priemerujú cez všetky vzorky.

Cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(p_{i,c} + \epsilon)$$

kde:

- N je počet vzoriek,
- C je počet tried,
- $y_{i,c}$ je 1, ak je trieda c správna pre vzorku i , a 0 inak,
- $p_{i,c} + e - 8$ je pravdepodobnosť predikovaná modelom pre triedu c pre vzorku i . $+ e-8$ sme pridali pre prípad keby vyšlo $\log(0)$

Dopredné šírenie pre skrytú vrstvu:

$$Z = WA + b$$

$$A = \text{ReLU}(Z)$$

Dopredné šírenie pre výstupnú vrstvu:

$$Z = WA + b$$

$$A = \text{softmax}(Z)$$

Softmax funkcia pre výstupnú vrstvu:

$$\text{softmax}(z) = \frac{e^z}{\sum_{k=1}^C e^z}$$

5 Spätné šírenie

Spätné šírenie sa začína výpočtom chyby pomocou cross-entropy medzi predikovanými a skutočnými hodnotami. Gradient tejto chyby s ohľadom na výstupy modelu sa vypočíta a použije na aktualizáciu váh a biasov výstupnej vrstvy pomocou gradientného zostupu. Potom sa tento gradient šíri späť cez skryté vrstvy, kde sa zohľadňuje derivácia aktivačnej funkcie (ReLU) a príslušné váhy. Na konci sa aktualizujú váhy a biasy všetkých vrstiev siete. Tento proces sa opakuje počas trénovania, čím sa model učí znižovať svoju chybu.

6 Trénovanie modelu

6.1 Nastavenie parametrov

Najprv sme zostavili základný model iba s jednou skrytou vrstvou, ktorý sme trénovali na 1 000 záznamoch a testovali na 200 - snažili sme sa dosiahnuť nejakú akceptovateľnú accuracy. Už v tomto bode sme videli, že naša architektúra lepšie funguje s aktivačnou funkciou relu (oproti sigmoid), a

taktiež sme zvolili aj He inicializáciu váh, ktorá nám pekne zvýšila accuracy. Tiež sme si všimli, že lepšie výsledky máme pri nižšom learning rate, a to $lr=0.01$.

Následne sme naše riešenie ešte rozšírili o viacero skrytých vrstiev. Zhodnotili sme, že 3 skryté vrstvy so 128, 64, 32 neurónmi fungujú najlepšie.

Následne sme ešte implementovali early stopping s parametrami **learning_rate=0.01**, **patience=20**, **min_delta=0.001**.

6.2 Priebeh tréovania

Počas tréovania sme si každých 100 epoch vypísali train accuracy, aby sme vedeli odsledovať accuracy voči test dátam kvôli preučeniu. Early stopping so správnymi parametrami nám tiež potenciálne pomáha zastaviť učenie včas - predtým, než sa model preučí.

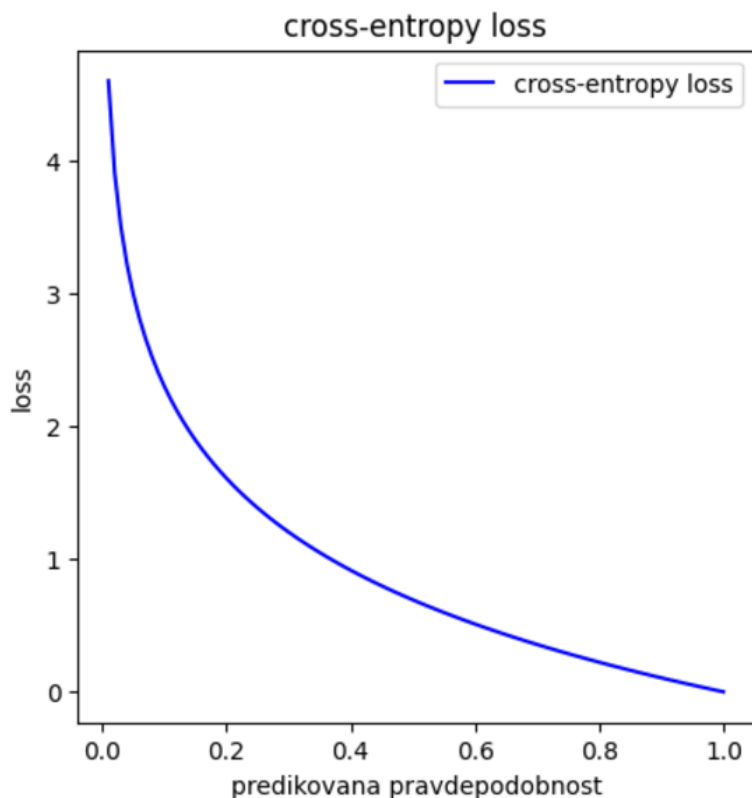


Figure 2: Graf straty modelu počas tréovania

```

epocha: 0, loss: 2.443332411152243, train acc: 10.47%
epocha: 100, loss: 1.1854542406725705, train acc: 64.92666666666666%
epocha: 200, loss: 0.8540438388262342, train acc: 71.47%
epocha: 300, loss: 0.7333794469823606, train acc: 75.30666666666667%
epocha: 400, loss: 0.6663681750422384, train acc: 77.735%
epocha: 500, loss: 0.6216352593285934, train acc: 79.25166666666667%
epocha: 600, loss: 0.5893927789151698, train acc: 80.20166666666667%
epocha: 700, loss: 0.5651531076105579, train acc: 80.97166666666666%
epocha: 800, loss: 0.5463516879292903, train acc: 81.44666666666667%
epocha: 900, loss: 0.5311907277655381, train acc: 81.89333333333333%
epocha: 1000, loss: 0.5185953083057847, train acc: 82.27333333333333%
epocha: 1100, loss: 0.5078542898982608, train acc: 82.515%
epocha: 1200, loss: 0.498533555114886, train acc: 82.79833333333333%
epocha: 1300, loss: 0.49029367090662096, train acc: 83.05166666666666%
epocha: 1400, loss: 0.4829448098507125, train acc: 83.22500000000001%
epocha: 1500, loss: 0.47631080970632866, train acc: 83.43333333333334%
epocha: 1600, loss: 0.4702596962020867, train acc: 83.61666666666666%
epocha: 1700, loss: 0.4647198436808987, train acc: 83.80833333333332%
Early stopping v epoche: 1790 s training loss: 0.46010125014713643
test acc: 83.97%
final train accuracy: 83.92500000000001%

```

Figure 3: Príklad výpisu po trénoch

7 Vyhodnotenie

Na odpoveď stačia 1-2 vety. Popíšte metriky, ktoré ste použili na vyhodnotenie výkonnosti modelu, ako je presnosť, a pridajte graf presnosti v priebehu tréovania.

Model sme vyhodnocovali najmä na základe toho, akú accuracy dosahoval počas tréovania, a akú mal na testovacích dátach. Ako je vidno aj na obrázkoch v predchádzajúcej kapitole, model sa aj napriek vysokému počtu epoch nepreučí. Bližšie sa môžeme pozrieť na úspešnosť výsledného modelu na graf, ktorý zobrazuje train loss a train accuracy. Ďalej confusion matrix konkrétne ukazuje, koľko a ktoré labels označil model správne a ktoré nie. Nakoniec môžeme zhodnotiť aj ROC krivku a poslednú iteráciu s aktivačnými hodnotami pre grafy softmax a relu aktivačných funkcií.

7.1 Otestovanie na vlastnom obrázku

Model sme nakoniec otestovali aj na obrázku ktorý sme našli na internete a následne predspracovali tak, aby s ním vedela pracovať neurónová sieť. Na obrázkoch môžeme vidieť obrázok pred a po spracovaní, a následnú klasifikáciu modelu. Model obrázok vyhodnotil ako triedu 0, čo je t-shirt, teda tričko na krátky rukáv, so 77,39% pravdepodobnosťou.

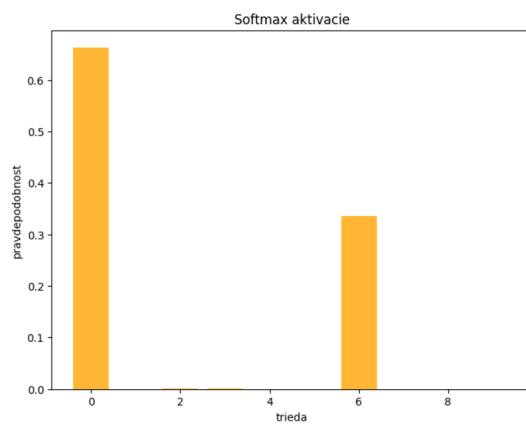


Figure 4: Hodnoty softmax pri poslednej iterácii

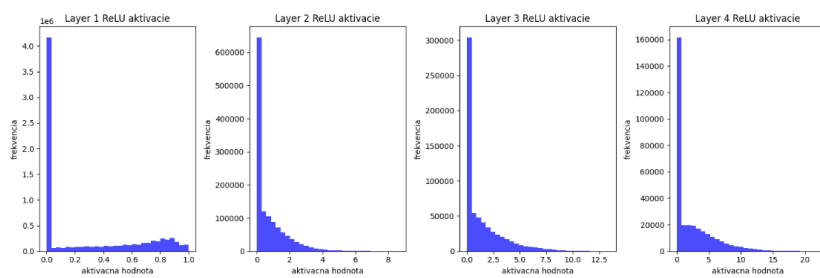


Figure 5: Hodnoty relu pri poslednej iterácii

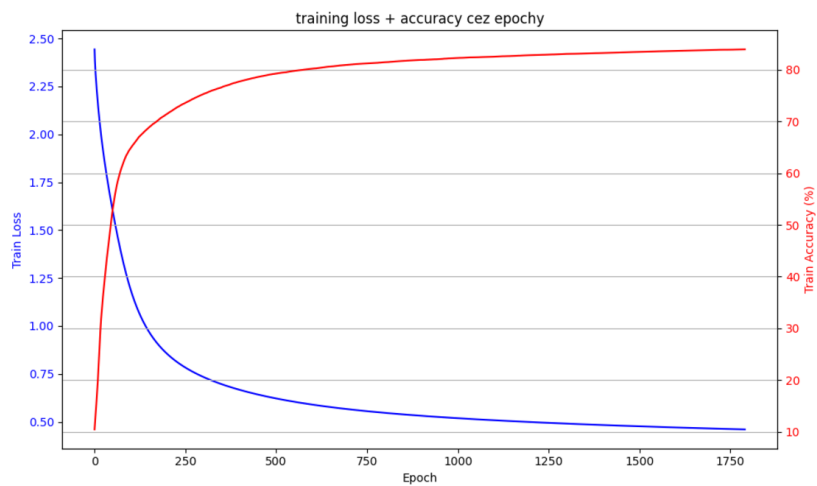


Figure 6: Accuracy pri tréovaní počas epoch

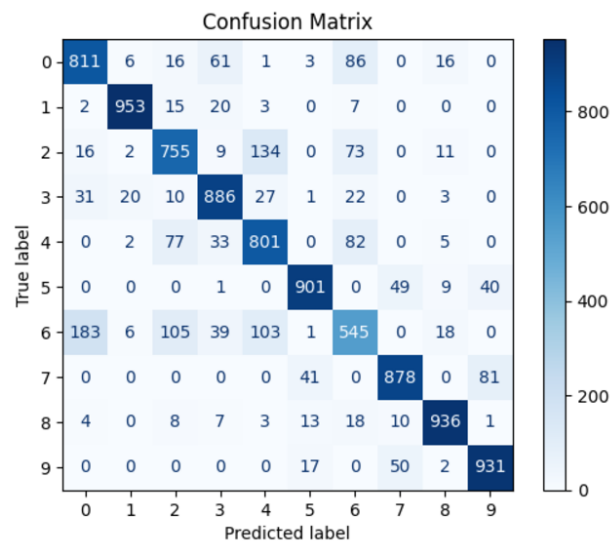


Figure 7: Confusion matrix po trénovaní

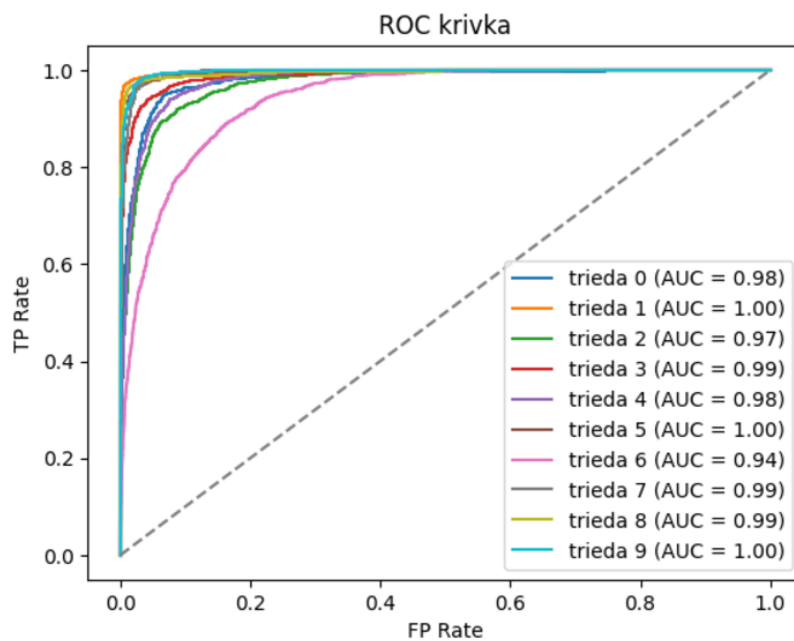


Figure 8: ROC krivka

8 Diskusia a zlepšenia

Aj napriek väčšej sieti a veľkému počtu epoch sa nám podarilo predísť preučeniu a celkovo dosiahnuť najlepšiu accuracy 85%. Model sa v tomto bode s našimi



Figure 9: Obrázok pred spracovaním

Predikovaná trieda: 0 s pravdepodobnosťou: 77.39%

Predikovaná trieda: 0 (77.39%)



Figure 10: Obrázok po spracovaní a klasifikácii

vedomostami asi už nedá viac vylepšiť. Každá z "nabalených" funkcionalít model zlepšila o nejaké 2-3%.

9 Záver

V tomto projekte sme navrhli a implementovali neurónovú sieť from scratch. Výslednému modelu, či už s jednou skrytou vrstvou alebo viacerými, sa po-

darilo dosiahnuť viac ako 50%, čím zadanie považujeme za splnené. Tak-
tiež sme vykreslili všetky príslušné grafy a otestovali úspešnosť modelu na
obrázku z internetu, ktorý sme predspracovali a následne dali modelu vyhod-
notiť.